

Solutionnaire du TP2

Partie A : les notes

1. Moyenne de E2 par année

```
grades |>
  group_by(Annee) |>
  summarise(moyenne_E2 = mean(E2))
#> # A tibble: 3 × 2
#>   Annee moyenne_E2
#>   <dbl>         <dbl>
#> 1  2018          66.9
#> 2  2019          74.4
#> 3  2020          87.7
```

2. Par année et par cohorte

```
grades |>
  group_by(Annee, Cohorte) |>
  summarise(n = n(), moyenne_E1 = mean(E1), moyenne_E2 = mean(E2))
#> `summarise()` has grouped output by 'Annee'. You can override using the
#> `groups` argument.
#> # A tibble: 6 × 5
#> # Groups:   Annee [3]
#>   Annee Cohorte      n moyenne_E1 moyenne_E2
#>   <dbl> <chr>    <int>      <dbl>      <dbl>
#> 1  2018 Cycle 1      16        69.3        55.7
#> 2  2018 Cycle 2 et 3  14        83.1        79.7
#> 3  2019 Cycle 1      14        69.1        69.4
#> 4  2019 Cycle 2 et 3  12        76.7        80.1
#> 5  2020 Cycle 1      11        73.5        84.8
#> 6  2020 Cycle 2 et 3  12        80.1        90.4
```

Le message rappelle que le résultat **reste groupé** par **Annee** (voir « Et si... ? », question 1).

3. Format long

```
grades_long <- grades |>
  pivot_longer(cols = -c(ID_etu, Annee, Cohorte), # toutes les colonnes sauf celles-ci
               names_to = "evaluation",
               values_to = "note")

grades_long
#> # A tibble: 474 × 5
#>   ID_etu Annee Cohorte evaluation  note
#>   <chr>  <dbl> <chr>   <chr>      <dbl>
#> 1 Etu_1  2018 Cycle 1 D1      90
#> 2 Etu_1  2018 Cycle 1 D2     87.5
#> 3 Etu_1  2018 Cycle 1 E1     94.4
#> 4 Etu_1  2018 Cycle 1 D3    100
#> 5 Etu_1  2018 Cycle 1 D4     98
#> 6 Etu_1  2018 Cycle 1 E2     92.9
#> 7 Etu_2  2018 Cycle 1 D1     86.7
#> 8 Etu_2  2018 Cycle 1 D2     88
#> 9 Etu_2  2018 Cycle 1 E1     83.3
#> 10 Etu_2 2018 Cycle 1 D3    100
#> # i 464 more rows
nrow(grades_long)
#> [1] 474
nrow(grades) * 6 # 79 étudiants x 6 évaluations
#> [1] 474
```

On aurait aussi pu écrire `cols = D1:E2` ou `cols = c(D1, D2, E1, D3, D4, E2)`.

4. Moyenne de toutes les évaluations par année

```
grades_long |>
  group_by(Annee) |>
  summarise(moyenne = mean(note)) |>
  arrange(desc(moyenne))
#> # A tibble: 3 × 2
#>   Annee moyenne
#>   <dbl>   <dbl>
#> 1  2019    83.2
#> 2  2018    81.6
#> 3  2020     NA
```

La moyenne de 2020 vaut `NA` : un étudiant de 2020 n'a pas de note au devoir 2 (voir TP1). Un seul `NA` rend la moyenne inconnue, et `arrange()` place le `NA` en dernier, ce qui fait croire que 2020 est la moins bonne année. Correction :

```
grades_long |>
  group_by(Annee) |>
  summarise(moyenne = mean(note, na.rm = TRUE)) |>
  arrange(desc(moyenne))
#> # A tibble: 3 × 2
```

```
#> Annee moyenne
#> <dbl> <dbl>
#> 1 2020 89.6
#> 2 2019 83.2
#> 3 2018 81.6
```

2020 est en fait la **meilleure** année.

5. Une ligne par évaluation, une colonne par année

```
grades_long |>
  summarise(moyenne = mean(note, na.rm = TRUE), .by = c(evaluation, Annee)) |>
  pivot_wider(names_from = Annee, values_from = moyenne)
#> # A tibble: 6 × 4
#>   evaluation `2018` `2019` `2020`
#>   <chr>      <dbl> <dbl> <dbl>
#> 1 D1        85.2  93.3  92.8
#> 2 D2        82.5  83.9  92.5
#> 3 E1        75.8  72.6  76.9
#> 4 D3        93.9  93.8  96.9
#> 5 D4        85.4  80.9  91.0
#> 6 E2        66.9  74.4  87.7
```

Partie B : les prénoms

6. Importer

```
baby <- read_csv("/home/public/tidyverse/babynames.csv")
```

```
#> Rows: 1924665 Columns: 5
#> — Column specification —————
#> Delimiter: ","
#> chr (2): sex, name
#> dbl (3): year, n, prop
#>
#> i Use `spec()` to retrieve the full column specification for this data.
#> i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
glimpse(baby)
#> Rows: 1,924,665
#> Columns: 5
#> $ year <dbl> 1880, 1880, 1880, 1880, 1880, 1880, 1880, 1880, 1880, 1880, 1880, ...
#> $ sex <chr> "F", "F", "F", "F", "F", "F", "F", "F", "F", "F", "F", "F", "F", ...
#> $ name <chr> "Mary", "Anna", "Emma", "Elizabeth", "Minnie", "Margaret", "Ida", ...
```

```
#> $ n      <dbl> 7065, 2604, 2003, 1939, 1746, 1578, 1472, 1414, 1320, 1288, 1258,...  
#> $ prop <dbl> 0.07238359, 0.02667896, 0.02052149, 0.01986579, 0.01788843, 0.016...
```

7. James en 1992

```
baby |> filter(sex == "M", name == "James", year == 1992)  
#> # A tibble: 1 × 5  
#>   year sex   name      n  prop  
#>   <dbl> <chr> <chr> <dbl> <dbl>  
#> 1  1992 M     James 28503 0.0136
```

8. James entre 1992 et 2001

```
baby |>  
  filter(sex == "M", name == "James", year %in% 1992:2001) |>  
  summarise(total = sum(n))  
#> # A tibble: 1 × 1  
#>   total  
#>   <dbl>  
#> 1 217096  
# équivalent : year >= 1992, year <= 2001
```

9. Le prénom le plus donné en 2000

```
baby |>  
  filter(year == 2000) |>  
  slice_max(n, n = 1)  
#> # A tibble: 1 × 5  
#>   year sex   name      n  prop  
#>   <dbl> <chr> <chr> <dbl> <dbl>  
#> 1  2000 M     Jacob 34471 0.0165  
# équivalent : arrange(desc(n)) |> head(1)
```

10. Filles ou garçons en 2001 ?

```
baby |>  
  filter(year == 2001) |>  
  summarise(total = sum(n), .by = sex)  
#> # A tibble: 2 × 2  
#>   sex      total  
#>   <chr>   <dbl>  
#> 1 F      1799636  
#> 2 M      1941815
```

Et si... ?

1. `part = n / sum(n)` après un `summarise()` à deux groupes

```
grades |>
  group_by(Annee, Cohorte) |>
  summarise(n = n()) |>
  mutate(part = n / sum(n))
#> `summarise()` has grouped output by 'Annee'. You can override using the
#> `.groups` argument.
#> # A tibble: 6 × 4
#> # Groups:   Année [3]
#>   Année Cohorte      n part
#>   <dbl> <chr>    <int> <dbl>
#> 1  2018 Cycle 1      16 0.533
#> 2  2018 Cycle 2 et 3  14 0.467
#> 3  2019 Cycle 1      14 0.538
#> 4  2019 Cycle 2 et 3  12 0.462
#> 5  2020 Cycle 1      11 0.478
#> 6  2020 Cycle 2 et 3  12 0.522
```

Le résultat est encore groupé par `Annee` : `sum(n)` est le nombre d'étudiants **de l'année**, et `part` est la part de chaque cohorte **dans son année**. La somme des `part` vaut 3 (1 par année), et non 1. Pour la part dans l'ensemble du cours, on retire le groupement :

```
grades |>
  group_by(Annee, Cohorte) |>
  summarise(n = n(), .groups = "drop") |>
  mutate(part = n / sum(n))
#> # A tibble: 6 × 4
#>   Année Cohorte      n part
#>   <dbl> <chr>    <int> <dbl>
#> 1  2018 Cycle 1      16 0.203
#> 2  2018 Cycle 2 et 3  14 0.177
#> 3  2019 Cycle 1      14 0.177
#> 4  2019 Cycle 2 et 3  12 0.152
#> 5  2020 Cycle 1      11 0.139
#> 6  2020 Cycle 2 et 3  12 0.152
```

Aucune des deux versions n'est fautive en soi : c'est la **question** qui dit laquelle est la bonne. L'erreur est de ne pas savoir laquelle on a calculée.

2. `mutate()` au lieu de `summarise()`

```
grades |>
  group_by(Annee) |>
  mutate(moyenne_E2 = mean(E2)) |>
```

```
select(ID_etu, Annee, E2, moyenne_E2)
#> # A tibble: 79 × 4
#> # Groups:   Annee [3]
#>   ID_etu Annee    E2 moyenne_E2
#>   <chr>  <dbl> <dbl>      <dbl>
#> 1 Etu_1   2018  92.9        66.9
#> 2 Etu_2   2018   90         66.9
#> 3 Etu_3   2018  30.7        66.9
#> 4 Etu_4   2018  82.9        66.9
#> 5 Etu_5   2018   5.71       66.9
#> 6 Etu_6   2018  34.3        66.9
#> 7 Etu_7   2018  48.6        66.9
#> 8 Etu_8   2018  76.4        66.9
#> 9 Etu_9   2018  81.4        66.9
#> 10 Etu_10 2018  25.7        66.9
#> # i 69 more rows
```

79 lignes, comme au départ : chaque étudiant reçoit la moyenne **de son année**. C'est utile pour comparer chaque note à la moyenne du groupe (**E2 - moyenne_E2**).

3. Pivoter toutes les colonnes

```
pivot_longer(grades, everything(), names_to = "evaluation", values_to = "note")
#> Error in `pivot_longer()` :
#> ! Can't combine `ID_etu` <character> and `D1` <double>.
```

Une erreur : la colonne **note** devrait contenir à la fois du texte (**ID_etu**, **Cohorte**) et des nombres. Une colonne n'a qu'un seul type. Les colonnes d'identification doivent rester des colonnes.

4. **n()** au lieu de **sum(n)**

```
baby |>
  filter(sex == "M", name == "James", year %in% 1992:2001) |>
  summarise(lignes = n(), bebes = sum(n))
#> # A tibble: 1 × 2
#>   lignes bebes
#>   <int> <dbl>
#> 1     10 217096
```

n() compte les **lignes** (une par année, donc 10), et non les **bébés**. La colonne s'appelle aussi **n**, ce qui entretient la confusion : **n()** est une fonction, **n** est une colonne.

5. **count(sex)** pour comparer filles et garçons

```
baby |> filter(year == 2001) |> count(sex)
#> # A tibble: 2 × 2
```

```
#>   sex      n
#>   <chr> <int>
#> 1 F      17970
#> 2 M      12299
```

`count()` compte les **lignes**, c'est-à-dire les **prénoms différents** : il y a plus de prénoms de filles que de prénoms de garçons. Mais il est né plus de garçons (question 10). La conclusion est **inversée**, sans aucun message. Toujours se demander ce que représente une ligne du tableau.

